

[파이썬 트랙] 1회차 월말평가 - Python



시험 과목

✓ Python

시험 목적

✓ Python 프로그래밍 기본 문법에 대한 이해

시험 유의 사항

- 1) 성실하게 테스트에 임할 것 (부정 행위시 성적 무효 및 중도 퇴소 처리)
- 2) 오픈소스 무단 복제 및 생성형 AI(chatGPT 등)를 활용한 부정행위 적발 시 0점 처리 또는 퇴실 조치될 수 있음
- 3) 각 문제별로 스켈레톤 코드가 작성된 파일이 제공되며, 해당 코드 파일을 수정하여 정답 코드를 작성할 것 (단, 주어진 함수명은 수정불가)
- 4) 소스코드 유사도 판단 프로그램 기준 부정행위로 판단될 시, 0점 처리 및 학사 기준에 의거 조치 실시 예정
- 5) 테스트 케이스와는 별도로 채점 케이스가 존재함
- 6) 내장 함수 활용 여부에 따라 배점 방식이 상이하므로 문제별 지시 사항을 필독할 것
- 7) 특정 내장 함수 활용 시 기본 점수 부여, 직접 로직 구현 시 가산점 부여
- 8) 특정 기능 사용 금지가 명시된 문항에서 해당 기능 사용 시 감점 처리
- 9) 사용자의 입력을 받는 input 함수는 절대 사용 금지
- 10) 요구하는 형식과 다른 정답을 반환할 시 오답으로 간주함

시험 코드 작성 유의 사항

최종 제출 코드가 다음 항목에 해당하는 경우, 감점 혹은 0점 처리 될 수 있음

- 1) Syntax Error로 인한 채점이 불가능한 경우
- 2) input 함수를 사용하는 경우
- 3) 주석 설명이 없거나 미흡한 경우
- 4) 출력 결과에 정답과 무관하거나 불필요한 내용이 있는 경우

※ 문제를 풀지 못하였다면 작성한 코드를 주석 처리하거나 삭제 후 pass 키워드 작성

※ 본 평가는 자동 채점 환경에서 채점이 이루어지므로, 상기 유의사항을 철저히 준수할 것

[파이썬 트랙] 1회차 월말평가 - Python



시험 환경

- Python 3.11
- 표준 파이썬 라이브러리 외 사용 금지
- Visual studio code(이하 vscode)를 이용한다.
- 그 외 (jupyter notebook, Pycharm, ...) 사용 불가

코드 실행

- vscode의 터미널창에서 python 실행 명령어로 결과 확인을 추천

```
Edwin@LECTURE MINGW64 /d/SSAFY  
$ python problem01.py
```

정답 제출 안내

제출 안내 사항 미 준수 시, 감점 혹은 0점 처리 될 수 있음

1) 압축 및 제출 파일 이름

- 지역_0반_홍길동

ex) 서울_1반_홍길동 / 부울경_2반_김싸피

2) 압축 폴더 구조

- 시험을 진행했던 폴더 구조 그대로
압축하여 제출 진행
- 우측 구조에서 '서울_1반_김싸피' 폴더 선택 후 압축

```
서울_1반_김싸피/  
    problem01.py  
    problem02.py  
    problem03.py  
    ...
```

제출 마감시간에 서버 요청이 집중될 수 있으므로, 미리 제출하는 것을 권장함
(마감 시간 이후 제출하지 않도록 최소 제출 마감 5분전 제출할 것)

[파이썬 트랙] 1회차 월말평가 - Python



문제 01 (problem01.py) – 9점

- ❖ 쇼핑몰 리뷰 분석 시스템을 개발 중인 김싸피는 상품에 달린 별점 데이터를 분석하려고 합니다.
- ❖ 한 상품에 달린 별점이 리스트로 주어질 때, **평균 별점을 실수(float) 형태로 반환하는 calculate_avg 함수를 완성**하시오.
- ❖ **별점 리스트에는 최소 1개 이상의 정수가 담겨 있습니다.**
- ❖ **결과값은 별도의 반올림 처리 없이 반환합니다.**
- ❖ 예시_01)
입력: [5, 4, 3, 5, 3]
출력: 4.0
- ❖ 예시_02)
입력: [4, 4, 4]
출력: 4.0
- ❖ 예시_03)
입력: [5, 4, 4]
출력: 4.333333333333333
- ❖ **제한 내장 함수: len, sum**
- ❖ **기본 점수 (9점): 제한 내장 함수를 사용한 해결**
- ❖ **가산점(+3점): 제한 내장 함수 없이 직접 구현 (총 12점)**

[파이썬 트랙] 1회차 월말평가 - Python



문제 02 (problem02.py) – 9점

- ❖ 커뮤니티 서비스의 닉네임을 점검하고 있습니다. 회원 닉네임 리스트 중에서, 특정 길이(N) 이상인 닉네임의 개수를 반환하는 `count_long_names` 함수를 완성하십시오.
- ❖ 닉네임 리스트는 문자열로 구성되어 있습니다.
- ❖ 기준 길이 `min_length`는 양의 정수입니다.
- ❖ 예시_01)
입력: ['kim', 'developer', 'ssafy', 'a'], 5
출력: 2 ('developer', 'ssafy'가 길이 5 이상)
- ❖ 예시_02)
입력: ['a', 'bb', 'ccc'], 5
출력: 0
- ❖ 제한 내장 함수: `len`
- ❖ 기본 점수 (9점): 제한 내장 함수를 사용한 해결
- ❖ 가산점(+3점): 제한 내장 함수 없이 직접 구현 (총 12점)

[파이썬 트랙] 1회차 월말평가 - Python



문제 03 (problem03.py) – 9점

- ❖ 김싸피는 음악 스트리밍 앱의 '최근 재생 목록 거꾸로 보기' 기능을 구현하고 있습니다.
- ❖ 재생한 곡 제목이 재생한 시간 순서대로(과거→최신) 리스트에 저장되어 있을 때, 이를 최신순(최신→과거)으로 보여주기 위해 리스트의 순서를 거꾸로 뒤집어 반환하는 `reverse_list` 함수를 완성하시오.
- ❖ 단, 재생 기록이 없는 경우 빈 리스트가 주어질 수 있습니다.
- ❖ 예시_01)
입력: [1, 2, 3, 4, 5]
출력: [5, 4, 3, 2, 1]
- ❖ 예시_02)
입력: ['Song A', 'Song B', 'Song C']
출력: ['Song C', 'Song B', 'Song A']
- ❖ 제한 Python 내장 함수: `reversed`, `list.reverse`
- ❖ 기본 점수 (9점): 제한 내장 함수를 사용한 해결
- ❖ 가산점(+3점): 제한 내장 함수 없이 직접 구현 (총 12점)

[파이썬 트랙] 1회차 월말평가 - Python



문제 04 (problem04.py) – 9점

- ❖ 특정 상품의 일별 가격 변동을 분석하려 합니다. 하루 단위로 기록된 가격이 리스트로 주어질 때, **최고가와 최저가의 차이(가격 변동폭)**를 정수로 반환하는 **find_price_range** 함수를 완성하시오.
- ❖ **가격 리스트에는 최소 1개 이상의 정수가 담겨 있습니다.**
- ❖ **가격이 1개뿐이면 변동폭은 0입니다.**
- ❖ 예시_01)
입력: [1000, 2500, 1800, 3000, 2200]
출력: 2000 (최고가 3000 - 최저가 1000)
- ❖ 예시_02)
입력: [500]
출력: 0
- ❖ **제한 내장 함수: max, min**
- ❖ 기본 점수 (9점): **제한 내장 함수**를 사용한 해결
- ❖ 가산점(+3점): **제한 내장 함수**없이 직접 구현 (총 12점)



문제 05 (problem05.py) – 9점

- ❖ 게임 캐릭터의 인벤토리를 정리하고 있습니다. 인벤토리는 여러 개의 가방(행)으로 나뉘어 있고, 각 칸에는 아이템 ID(0이 아닌 정수)가 들어 있으며 빈 칸은 0으로 표시됩니다.
- ❖ 가방별 칸 수가 다를 수 있는 2차원 리스트 **inventory**가 주어질 때, 빈 칸(0)을 제외한 **실제 아이템이 들어 있는 칸의 총개수**를 반환하는 **count_items** 함수를 완성하시오.

❖ 예시_01)

입력: [[101, 0, 205], [33, 4, 5], [0]]

출력: 5

❖ 예시_02)

입력: [[0, 0], [0], [88, 90]]

출력: 2

- ❖ **제한 내장 함수: sum, len, map**
- ❖ **기본 점수 (9점): 제한 내장 함수를 사용한 해결**
- ❖ **가산점(+3점): 제한 내장 함수 없이 직접 구현 (총 12점)**

[파이썬 트랙] 1회차 월말평가 - Python



문제 06 (problem06.py) – 9점

- ❖ 여러 센서 그룹의 측정값을 분석합니다. $N \times N$ 2차원 정수 행렬이 주어질 때, 각 행(그룹)의 최댓값들을 모두 더한 값을 반환하는 `sum_row_maximums` 함수를 완성하시오.
- ❖ 행렬의 원소는 절댓값이 10,000 이하인 정수입니다. (음수 포함 가능)
- ❖ 행렬의 크기 N 은 1 이상 100 이하의 정수입니다.
- ❖ 예시_01)
입력: `[[10, 3, 7], [2, 9, 4], [6, 1, 8]]`
출력: 27 (10 + 9 + 8)
- ❖ 예시_02)
입력: `[[-1, -2, -3], [-10, -5, -1], [-100, -200, -300]]`
출력: -102 ((-1) + (-1) + (-100))
- ❖ 예시_03)
입력: `[[5]]`
출력: 5
- ❖ 제한 내장 함수: `map`, `max`, `sum`
- ❖ 기본 점수 (9점): 제한 내장 함수를 사용한 해결
- ❖ 가산점(+3점): 제한 내장 함수 없이 직접 구현 (총 12점)

[파이썬 트랙] 1회차 월말평가 - Python



문제 07 (problem07.py) – 7점

- ❖ 인사 담당자 김싸피는 사원 명단을 부서별로 정리하는 프로그램을 작성했습니다. 그런데 실행 결과가 이상하게 나와 원인을 찾고 있습니다.
- ❖ 사원 정보가 담긴 딕셔너리 리스트({'name': ..., 'dept': ...})가 주어질 때, 부서(dept)를 키(key)로 하고 해당 부서에 속한 사원 이름(name)의 리스트를 값(value)으로 하는 딕셔너리를 반환해야 하는 group_by_dept 함수입니다.
- ❖ 여러 부서가 섞여 있을 때 결과가 잘못 나오는 원인을 찾아 함수를 수정하십시오. (함수명은 수정 불가)
- ❖ 사원 정보가 없는 경우 빈 리스트가 주어질 수 있습니다.

❖ 예시)

입력: employees = [

```
{'name': 'Kim', 'dept': 'Dev'},
{'name': 'Lee', 'dept': 'Dev'},
{'name': 'Park', 'dept': 'HR'},
{'name': 'Choi', 'dept': 'Sales'}
```

]

❖ 현재 코드 실행 결과 (모든 부서가 같은 명단으로 뒤섞여 나옴):

```
{ 'Dev': ['Kim', 'Lee', 'Park', 'Choi'],
  'HR': ['Kim', 'Lee', 'Park', 'Choi'],
  'Sales': ['Kim', 'Lee', 'Park', 'Choi'] }
```

❖ 올바른 출력(기댓값):

```
{ 'Dev': ['Kim', 'Lee'],
  'HR': ['Park'],
  'Sales': ['Choi'] }
```

[파이썬 트랙] 1회차 월말평가 - Python



문제 08 (problem08.py) – 7점

- ❖ 두 데이터셋의 교집합을 구하려 합니다.
- ❖ 두 개의 리스트 `list_a`, `list_b`가 주어질 때, 두 리스트에 모두 존재하는 원소를 `list_a`에 처음 등장한 순서대로, 중복 없이 담은 리스트를 반환하는 `find_common` 함수를 완성하십시오.

- ❖ 각 리스트에는 중복된 원소가 들어 있을 수 있습니다.
- ❖ 공통 원소가 없으면 빈 리스트를 반환합니다.

❖ 예시_01)

입력: [1, 2, 3, 4, 5], [2, 4, 6, 4]

출력: [2, 4]

❖ 예시_02)

입력: ['apple', 'banana', 'cherry'], ['cherry', 'apple', 'grape']

출력: ['apple', 'cherry'] (list_a 순서 기준)

[파이썬 트랙] 1회차 월말평가 - Python



문제 09 (problem09.py) – 7점

- ❖ 김싸피는 창고의 재고 관리 시스템을 구현하고 있습니다.
- ❖ 창고의 초기 재고가 {품목명: 수량} 형태의 딕셔너리로 주어지고, 입출고 명령이 (품목명, 변화량) 튜플의 리스트로 주어집니다.
- ❖ 모든 명령을 순서대로 처리한 후의 (최종 재고 딕셔너리, 거부된 출고 명령 수)를 튜플로 반환하는 `manage_inventory` 함수를 완성하시오.
- ❖ [처리 규칙]
 - 변화량이 양수이면 입고: 해당 품목의 재고를 증가시킵니다. 초기 재고에 없던 품목이면 새 품목으로 추가합니다.
 - 변화량이 음수이면 출고: 해당 품목의 재고에서 그만큼 차감합니다.
 - 재고 부족 시 출고 거부: 출고하려는 수량이 현재 재고보다 많으면, 해당 명령은 무시(재고 변화 없음)하고 거부 횟수를 1 증가시킵니다. (재고는 절대 음수가 될 수 없음)
 - 초기 재고에 없던 품목을 출고하려는 경우, 현재 재고를 0으로 간주하여 거부 처리합니다.

❖ 예시_1)

입력:

```
initial = {'apple': 10, 'banana': 5}
```

```
commands = [('apple', -3), ('banana', -8), ('cherry', 4), ('apple', 5), ('banana', -2)]
```

출력: ({'apple': 12, 'banana': 3, 'cherry': 4}, 1)

처리 과정:

- ('apple', -3): 재고 10 → 출고 성공 → apple = 7
- ('banana', -8): 재고 5 < 8 → 거부 (거부 1)
- ('cherry', 4): 신규 입고 → cherry = 4
- ('apple', 5): 입고 → apple = 12
- ('banana', -2): 재고 5 ≥ 2 → 출고 성공 → banana = 3

[파이썬 트랙] 1회차 월말평가 - Python



문제 09 (problem09.py) – 7점

❖ 예시_2)

입력:

```
initial = {}  
commands = [('pen', -1), ('pen', 10), ('pen', -3)]
```

출력: ({'pen': 7}, 1)

처리 과정:

- ('pen', -1): 재고 0 $\rightarrow$ 거부 (거부 1)
- ('pen', 10): 신규 입고 $\rightarrow$ pen = 10
- ('pen', -3): 재고 $10 \geq 3 \rightarrow$ 출고 성공 $\rightarrow$ pen = 7

[파이썬 트랙] 1회차 월말평가 - Python



문제 10 (problem10.py) – 7점

- ❖ 자율비행 드론이 격자 형태의 구역을 비행합니다.
- ❖ 구역은 2차원 리스트 **grid**로 주어지며, 값이 **0**이면 비행 가능, **1**이면 장애물입니다.
- ❖ 드론은 시작 위치 **start = (row, col)**에서 출발하여 처음에는 위쪽(상)을 바라봅니다.
- ❖ 이동 명령 문자열 **commands**가 주어질 때, 모든 명령을 수행한 후 드론의 최종 위치 (**row, col**)를 튜플 형태로 반환하는 **simulate_drone** 함수를 완성하시오.

❖ [명령어 규칙]

- F (Forward): 현재 바라보는 방향으로 1칸 전진
- R (Right): 시계 방향으로 90도 회전 (방향만 바뀜)
- L (Left): 반시계 방향으로 90도 회전 (방향만 바뀜)

❖ [방향 및 좌표 규칙] (격자는 0-indexed, row는 위에서 아래로 증가)

- 상(Up): row - 1
- 하(Down): row + 1
- 좌(Left): col - 1
- 우(Right): col + 1

❖ [전진 제약]

- ❖ 전진하려는 칸이 격자 범위를 벗어나거나 장애물(1)이면, 그 전진 명령은 무시합니다.
(제자리 유지, 방향도 그대로)

❖ 예시_1)

다음 페이지에 계속

[파이썬 트랙] 1회차 월말평가 - Python



문제 10 (problem10.py) – 7점

- 입력: `grid = [[0, 0, 0],`
 `[0, 1, 0],`
 `[0, 0, 0]]`

`start = (0, 0)`

`commands = "FRFF"`

- 출력: `(0, 2)`

- 처리과정:

- 시작 (0,0), 상 → F: 위는 격자 밖 → 무시 (0,0 유지)
- R회전 → 우를 봄
- F: (0,1) 비행 가능 → (0,1)
- F: (0,2) 비행 가능 → (0,2)

❖ 예시_2)

- 입력: `grid = [[0, 0, 0],`
 `[0, 1, 0],`
 `[0, 0, 0]]`

`start = (2, 1)`

`commands = "FRFF"`

- 출력: `(2, 2)`

- 처리과정:

- 시작 (2,1), 상 → F: (1,1)은 장애물 → 무시 (2,1 유지)
- R회전 → 우를 봄
- F: (2,2) 비행 가능 → (2,2)
- F: (2,3) 격자 밖 → 무시 (2,2 유지)